

SPEC-006 — MEF Knowledge Engine (Machine-Readable Implementation Blueprint, Part 1)

October 4, 2025

Contents

1	Scope and Non-Destructive Contract	2
1.1	Purpose	2
1.2	Hard Constraints (MUST NOT)	2
1.3	Allowed Changes (ADD-ONLY)	2
1.4	Compatibility Policy (SHALL)	2
2	Determinism & Seed Governance (Invariant Set)	2
2.1	Seeds	2
2.2	Content Addressing	2
2.3	Reproducibility	2
3	Feature Flags & Safe Defaults	2
3.1	Config Additions (ADD-ONLY)	2
3.2	Behavior	3
4	Materializer & File Layout Contract	3
4.1	Flat Input Filenames (Agent Output)	3
4.2	Target Tree (Canonical)	3
4.3	Package Inits	3
5	S0 — Repository Scan & Non-Destructive Plan	3
5.1	Objective	3
5.2	Required Checks (SHALL)	3
5.3	Success Criteria	3
6	S1 — Schemas (ADD-ONLY)	3
6.1	Artifacts (MUST exist or be added)	3
6.2	Rules	4
7	S2 — API Attachment (ADD-ONLY)	4
7.1	Router Registration	4
7.2	Non-Interference	4
8	S3 — Config Additions (Safe Defaults)	4
8.1	YAML Contract (ADD-ONLY)	4
8.2	Validation	4

1 Scope and Non-Destructive Contract

1.1 Purpose

This document instructs an automated coding agent to extend MEF-Core with a Knowledge/Memory layer and related APIs **without** altering core logic. It is deterministic, machine-executable, and assumes an existing codebase that may *not* match this text 1:1.

1.2 Hard Constraints (MUST NOT)

- **MUST NOT** alter semantics of core operators DK, SW, PI, WT.
- **MUST NOT** change Solve-Coagula iteration rules, PoR decision logic, Merkaba-Gate thresholds, HDAG invariants.
- **MUST NOT** break existing JSON schemas; only additive fields are allowed in new artifacts.
- **MUST NOT** remove or rename existing public endpoints; new endpoints use new prefixes.
- **MUST NOT** write secrets (BIP-39 mnemonic) to logs or ledger.

1.3 Allowed Changes (ADD-ONLY)

- **MAY** add new modules under `src/knowledge/`, `src/api/`, `src/schemas/`.
- **MAY** add compatibility shims (e.g., `ingestion.triton`) to avoid refactors.
- **MAY** add new config keys with safe defaults (feature flags OFF).
- **MAY** include routers via `app.include_router(...)` under new prefixes.

1.4 Compatibility Policy (SHALL)

- **SHALL** prefer adapters/shims over edits to existing files.
- **SHALL** keep behavior identical when new features are disabled.
- **SHALL** pass a deterministic regression suite before/after integration.

2 Determinism & Seed Governance (Invariant Set)

2.1 Seeds

- Root seed is a BIP-39 mnemonic (never persisted). Sub-seeds are derived HD-style from path strings:

$$s_{\text{sub}} = \text{HMAC_SHA256}(s_{\text{root}}, \text{"MEF/" + \langle domain \rangle + \langle stage \rangle + \langle index \rangle"})$$

2.2 Content Addressing

- `mef_id = HASH( canonical(TIC) || route_id || seed_path )`, canonical JSON with stable key order and fixed float precision.

2.3 Reproducibility

- Identical inputs + identical seed path $\Rightarrow$ identical $\hat{z}$, route, TIC, ledger hash. Any deviation is a defect.

3 Feature Flags & Safe Defaults

3.1 Config Additions (ADD-ONLY)

- `knowledge.enabled = false` (default).
- `memory.enabled = false` (default).
- `router.mode  $\in$  {inproc, service}`, default `inproc`.
- `paths.memory` (filesystem path), default unset.

3.2 Behavior

- With all new flags **false**, runtime behavior **must equal** pre-integration behavior.
- Enabling flags only adds functionality; it never changes core outcomes.

4 Materializer & File Layout Contract

4.1 Flat Input Filenames (Agent Output)

- Use double-underscore path encoding: `src__package__file.py`.
- Fix `*.py.py` to `*.py`; replace dashes with underscores.

4.2 Target Tree (Canonical)

- `src/api`, `src/ingestion`, `src/spiral`, `src/solvecoagula`, `src/gates`, `src/knowledge`, `src/hdag`, `src/ledger`, `src/storage`, `src/schemas`

4.3 Package Inits

- Ensure `__init__.py` in every `src/*` subdirectory.

5 S0 — Repository Scan & Non-Destructive Plan

5.1 Objective

Detect actual module names/paths and install shims instead of editing core files.

5.2 Required Checks (SHALL)

1. Detect presence of the following imports in the existing codebase:
 - `ingestion.triton` $\vee$ `ingestion.triton_core`
 - `gates.merkaba_gate`
 - `metatron.` $\vee$ `metatron_vendor.`
2. If `ingestion.triton` is imported and `triton.py` is missing, **SHALL** create a shim:

```
triton.py: from .triton_core import normalize_payload, TritonCore as TritonNormalizer
```

3. Verify `src/api/server.py` exists and is the FastAPI entrypoint.
4. Verify schemas exist; do not overwrite; add only new files.

5.3 Success Criteria

- All imports resolve without editing legacy files.
- Running `uvicorn src.api.server:app` succeeds as before.

6 S1 — Schemas (ADD-ONLY)

6.1 Artifacts (MUST exist or be added)

- `src/schemas/spiral_snapshot.json`
- `src/schemas/tic.json`
- `src/schemas/mef_block.json`
- `src/schemas/merkaba_gate.json`
- **NEW:** `src/schemas/route_spec.json`
- **NEW:** `src/schemas/memory_item.json`

6.2 Rules

- Existing schema fields **SHALL NOT** be removed or renamed.
- New schemas **SHALL** use draft-07 and stable numeric formatting.

7 S2 — API Attachment (ADD-ONLY)

7.1 Router Registration

- In `src/api/server.py`, **SHALL** register new routers with new prefixes only:
 - `/router` (Metatron selection API, optional)
 - `/memory` (vector memory API, optional)
 - `/knowledge` (derive/project/validate, optional)

7.2 Non-Interference

- Existing endpoints remain unchanged. New endpoints **MUST NOT** alter shared state in a way that changes prior outputs when disabled.

8 S3 — Config Additions (Safe Defaults)

8.1 YAML Contract (ADD-ONLY)

- Keys:

```
knowledge:
  enabled: false
memory:
  enabled: false
router:
  mode: inproc    # or service
paths:
  memory: ""      # empty => local disabled
```

8.2 Validation

- If `enabled=false`, modules **MUST** no-op and not allocate indices.
- If `paths.memory` unset/empty, index **MUST** remain disabled.

Stop Condition for Part 1

Proceed to Part 2 only if:

- App boots unchanged with all new flags `false`.
- Import graph resolves with shims.
- Schemas present; no legacy file modified.

SPEC-006 — MEF Knowledge Engine (Machine-Readable Implementation Blueprint, Part 2)

October 4, 2025

1 Complete Mathematical Definitions (Normative)

1.1 Spaces and Symbols

- State vector $x \in \mathbb{R}^5$, coordinates $x = (x_1, \dots, x_5)$.
- Spectral signature $\sigma = (\psi, \rho, \omega) \in \mathbb{R}^3$.
- Weighted storage vector $z' \in \mathbb{R}^8$ and normalized $\hat{z} \in \mathbb{R}^8$.

1.2 Spiral(5D) Embedding (Deterministic)

Given seed s and phase $\theta \in \mathbb{R}$:

$$S(\theta) = (a \cos \theta, a \sin \theta, b \cos(2\theta), b \sin(2\theta), c \theta), \quad a, b, c > 0.$$

Sampling points $\theta_k = \theta_0 + k\Delta\theta$, $\Delta\theta = \frac{2\pi}{N}$, $k = 0, \dots, N-1$. The raw 5D vector is $x = S(\theta_k)$. All parameters are read from config and are *constant* for a run.

1.3 Spectral Signature (PoR features)

Let $x_{1:T} \in \mathbb{R}^{T \times 5}$ be a short window (deterministic length T) of recent 5D states. Compute per-dimension FFT power $P_j(f)$ and define bandpowers:

$$E_L = \sum_{f \in \mathcal{B}_L} \sum_{j=1}^5 P_j(f), \quad E_M = \sum_{f \in \mathcal{B}_M} \sum_{j=1}^5 P_j(f), \quad E_H = \sum_{f \in \mathcal{B}_H} \sum_{j=1}^5 P_j(f).$$

Define ratio features

$$\psi = \frac{E_M}{E_L + E_M + E_H}, \quad \rho = \frac{E_L}{E_M + E_H + \varepsilon}, \quad \omega = \frac{E_H}{E_L + E_M + \varepsilon},$$

with $\varepsilon > 0$ fixed. These are scale-invariant.

1.4 Proof of Resonance (PoR) Decision

Let λ_{gap} be the spectral gap of the normalized Laplacian $\mathcal{L}$ of a k -NN graph built on $\{x_t\}_{t=T-w+1}^T$ with fixed k, w :

$$\mathcal{L} = I - D^{-1/2} A D^{-1/2}, \quad \lambda_{\text{gap}} = \lambda_2 - \lambda_1.$$

Decision:

$$\text{PoR}(x_{1:T}, \sigma) = \text{valid} \iff \left(\psi \in [\psi_{\min}, \psi_{\max}] \right) \wedge \left(\lambda_{\text{gap}} \geq \lambda \right) \wedge \left(\text{SNR} \geq s \right).$$

All thresholds are in config; SNR computed deterministically from bandpowers.

1.5 Operators (Solve—Coagula, Non-Expansive)

Let $v \in \mathbb{R}^d$ be the working state (d defined by the implementation). The operator route is a 7-slot permutation over

$$\text{SLOTS} = \{\text{DK}, \text{SW}, \text{PI}, \text{WT}, \text{RES1}, \text{ADAPTER}, \text{RES2}\},$$

where only DK, SW, PI, WT are mandatory; the others are no-ops by default.

DK (DoubleKick).

$$v^{(1)} = v + \alpha_1 \text{clip}(Av, -c, c), \quad v' = v^{(1)} - \alpha_2 \text{clip}(Bv^{(1)}, -c, c),$$

with fixed $\alpha_1, \alpha_2, c > 0$, matrices A, B with $\|A\|_2 \leq \kappa_A$, $\|B\|_2 \leq \kappa_B$ and $\alpha_1 \kappa_A + \alpha_2 \kappa_B \leq 1$. The clip is 1-Lipschitz; thus DK is non-expansive.

SW (Soft-Threshold Sweep). Apply element-wise soft-thresholding:

$$\text{soft}_\tau(x) = \text{sign}(x) \cdot \max(|x| - \tau, 0), \quad v' = \text{soft}_\tau(v).$$

This is firmly non-expansive.

PI (Path Invariance Projection). Let $\mathcal{G}$ be the finite group of admissible permutations/sign-flips induced by symmetry (implementation constant). The canonical representative is

$$\Pi(v) = \arg \min_{g \in \mathcal{G}} \text{lex}(g \cdot v),$$

where lex is a deterministic lexicographic order. Then $v' = \Pi(v)$. PI is idempotent and 1-Lipschitz.

WT (Weight Transfer).

$$v' = (1 - \gamma)v + \gamma Wv, \quad \gamma \in [0, 1], \quad W \text{ row-stochastic}, \quad \|W\|_2 \leq 1.$$

Convex combination of non-expansive maps is non-expansive.

1.6 Solve-Coagula Iteration and Convergence

Let T be the composition of operators in the selected route. Use fixed-point iteration with averaging:

$$v_{t+1} = (1 - \lambda)v_t + \lambda T(v_t), \quad \lambda \in (0, 1).$$

This is the Krasnosel'skii-Mann scheme; with T non-expansive and constant $\lambda \in (0, 1)$ it converges to a fixed point v under standard conditions (bounded orbits). Deterministic stop:

$$\text{stop} \iff \|v_{t+1} - v_t\|_2 \leq \epsilon_{\text{fix}} \quad \text{or} \quad t \geq t_{\text{max}}.$$

1.7 Lyapunov Functional and Metrics

$$V(v) = \|T(v) - v\|_2^2, \quad \Delta V = V(v_{t+1}) - V(v_t).$$

Gate requires $\Delta V < 0$ at termination step.

1.8 Gate Metrics

$$\Delta PI = \|\Pi(v_{t+1}) - \Pi(v_t)\|_2, \quad \Phi = \frac{\langle v_{t+1}, T(v_t) \rangle}{\|v_{t+1}\|_2 \|T(v_t)\|_2}.$$

$$\text{FIRE} \iff (\text{PoR} = \text{valid}) \wedge (\Delta PI \leq \epsilon) \wedge (\Phi \geq \phi) \wedge (\Delta V < 0).$$

2 Deterministic Vector Construction & Distance

2.1 Weighted Augmentation

Let $w \in \mathbb{R}_{>0}^8$ be fixed weights from config. Build

$$z' = [w_1 x_1, \dots, w_5 x_5, w_\psi \psi, w_\rho \rho, w_\omega \omega] \in \mathbb{R}^8.$$

Normalize $\hat{z} = z' / \|z'\|_2$. Store only $\hat{z}$ for memory/indexing.

2.2 Similarity

Cosine on $\hat{z}$ equals L2 on $\hat{z}$:

$$\cos(\hat{z}, \hat{y}) = 1 - \frac{\|\hat{z} - \hat{y}\|_2^2}{2}.$$

Agents SHALL use either cosine or L2 on normalized vectors, deterministically.

3 Metatron Adapter (Deterministic Router)

3.1 S7 Permutation Space

Let $\mathfrak{S}_7$ be all permutations of $\{1, \dots, 7\}$ in lexicographic order: $\{\sigma_i\}_{i=1}^{5040}$.

3.2 Selection Rule

Given seed s and mesh metrics m (e.g., invariants), compute:

$$h = \text{SHA256}(s \parallel \text{canonical_json}(m)), \quad k = \lfloor 1000 |J(m)| \rfloor \bmod 5040, \quad \text{idx} = (h \bmod 5040 + k) \bmod 5040,$$

with a deterministic score $J(m) = \alpha_b \sum \beta + \alpha_\lambda \lambda_{\text{gap}} + \alpha_p \sum \text{pers}$. The selected permutation is $\sigma = \sigma_{\text{idx}}$. Map to SLOTS accordingly.

3.3 Ledger Proof

Persist $\text{route_id} = \text{SHA256}(s \parallel \sigma, \text{permutation (slot names)}, \text{and } J(m) \text{ inside the block proof. Add-only.}$

4 Knowledge/Memory Layer (Add-Only)

4.1 Knowledge Object

```
{
  "mef_id": "string",
  "tic": { "...": "TIC canonical content" },
  "proofs": { "por": "...", "phi": 0.0, "delta_pi": 0.0, "delta_v": -0.0,
               "route": {"route_id": "...", "sigma": [...], "permutation": [...]} },
  "seed_path": "MEF/<domain>/<stage>/<index>",
  "context": { "hdag_refs": [], "parents": [], "children": [] }
}
```

4.2 Memory Item (Vector Store)

```
{
  "id": "string",
  "vector8": [0,0,0,0,0,0,0,0], // stores hat{z}
  "sigma": {"psi": 0, "rho": 0, "omega": 0},
  "por": "valid",
  "tic_id": "string",
  "metadata": {}
}
```

4.3 API Contracts (OpenAPI-ready)

/memory/upsert (POST). Input: `MemoryItem`; MUST check `por==valid`; persist `vector8` and `metadata`.
Output: `"ok":true,"id":"..."`.

/memory/search (POST). Input: `"query_vector8": [...], "top_k": K, "filters": ;` computes kNN over normalized vectors; MAY rerank by Φ or ΔPI .
Output: `"hits": [{"id": "...", "score": 0.0, ...}]`.

/knowledge/derive (POST). Input: `"inputs": [...], "seed_path": "MEF/...", "config": .`
Procedure: run S4-S9; on FIRE, assemble Knowledge and append block.
Output: `"knowledge": ..., "tic": ..., "proof": ..., "ledger_block": N`.

/knowledge/project (POST). Input: `"mef_id": "...", "target_domain": "...", "policy": .`
Output: `"projection": ..., "proof": ....`

5 HDAG Update (Path Control)

Nodes reference TICs/MEFs; edges link temporal/semantic adjacency. Update rule:

append edge $(u \rightarrow v)$ iff $u \prec v \wedge \text{acyclic}(G \cup \{u \rightarrow v\})$.

Path invariance check: for parallel routes $\mathcal{P}_1, \mathcal{P}_2$ ending at same snapshot, enforce $\|\Pi(v_1) - \Pi(v_2)\|_2 \leq \epsilon$; else refuse edge.

6 Ledger Proof Bundle (Add-Only)

Each committed block MUST include:

$\{ \text{por}, \phi, \Delta pi, \Delta v, \text{route}, \text{snapshot_hash}, \text{tic_hash} \}$.

Hash function and canonical serialization SHALL be identical to existing code; if unknown, the agent MUST implement a canonical JSON (UTF-8, stable key order, fixed float precision) and keep existing code untouched.

7 Deterministic Tests (Acceptance)

7.1 Unit

- Spiral: fixed seed/path $\Rightarrow$ identical x and σ .
- Operators: DK/SW/PI/WT are 1-Lipschitz or averaged non-expansive.
- PoR: amplitude scaling of $x_{1:T}$ leaves decision unchanged.

7.2 Property

- Seed reproducibility: same seed/path $\Rightarrow$ identical $\hat{z}$, route, TIC, ledger hash.
- Cosine=L2 on $\hat{z}$ equivalence.

7.3 Integration

- S4-S9 end-to-end: Snapshot $\rightarrow$ Gate FIRE $\rightarrow$ TIC/Knowledge $\rightarrow$ Ledger append.
Repeat twice $\Rightarrow$ identical hashes.

Implementation Staircase (continue)

S4 — Spiral(5D) & PoR

Compute $x = S(\theta)$ and σ ; compute PoR ratios and λ_{gap} ; emit valid/invalid with metrics. MUST NOT change PoR thresholds if they already exist; if unknown, read from config defaults.

S5 — Operators & Iteration

Load the current non-expansive DK/SW/PI/WT from core; do not edit. Build route from Metatron Adapter. Run fixed-point iteration with λ and limits from config; record ΔV and ΔPI .

S6 — Merkaba-Gate

Evaluate FIRE rule using $\{\text{PoR}, \Delta PI, \Phi, \Delta V\}$; on HOLD, return event and skip commit; on FIRE, continue.

S7 — TIC Crystallization

Assemble TIC invariants and content hash using canonical JSON; MUST NOT rename fields; add-only if extension needed.

S8 — Knowledge Object

Bind TIC with seed path and route proof; compute mef_id; prepare Knowledge bundle.

S9 — Memory Upsert (optional)

If memory.enabled=true and path configured, persist $\hat{z}$ and metadata via local index wrapper; MUST be no-op if disabled.

S10 — HDAG Update

Append node/edges respecting acyclicity and path invariance. MUST NOT alter existing graph logic; add-only.

S11 — Ledger Append

Append new block with proof bundle; verify chain integrity; MUST NOT change block format; only add proof.metatron and proof.gate fields if absent.

S12 — APIs (Attach Only)

Register routers under new prefixes; zero changes to existing endpoints. Provide OpenAPI with schemas for route_spec, memory_item, knowledge.

SPEC-006 — MEF Knowledge Engine (Machine-Readable Implementation Blueprint, Part 3)

October 4, 2025

1 JSON Schemas (Normative, draft-07)

1.1 spiral_snapshot.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "SpiralSnapshot",
  "type": "object",
  "properties": {
    "id": {"type": "string"},
    "timestamp": {"type": "string", "format": "date-time"},
    "seed": {"type": "string"},
    "phase": {"type": "number"},
    "coordinates": {
      "type": "array", "items": {"type": "number"}, "minItems": 5, "
      maxItems": 5
    },
    "sigma": {
      "type": "object",
      "properties": {"psi":{"type":"number"}, "rho":{"type":"number"}, "
        omega":{"type":"number"}}},
      "required": ["psi","rho","omega"]
    },
    "metrics": {
      "type": "object",
      "properties": {"resonance":{"type":"number"}, "stability":{"type":"
        number"}, "por":{"type":"string","enum":["valid","invalid"]}},
      "required": ["resonance","stability","por"]
    },
    "payload": {"type": "object"},
    "hdag_node": {"type": "string"}
  },
  "required": ["id","timestamp","seed","phase","coordinates","sigma","
    metrics"]
}
```

1.2 tic.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "TIC",
  "type": "object",
  "properties": {
    "tic_id": {"type": "string"},
    "snapshot_id": {"type": "string"},
  }
}
```

```

    "delta_pi":{"type":"number"},
    "mci":{"type":"number"},
    "phi":{"type":"number"},
    "gate":{"type":"object","properties":{"por":{"type":"string","enum":["valid","invalid"]},"fire":{"type":"boolean"}}, "required":["por","fire"]},
    "invariants":{"type":"object","properties":{"symmetry":{"type":"string"},"entropy":{"type":"number"}}},
    "timestamp":{"type":"string","format":"date-time"}
  },
  "required":["tic_id","snapshot_id","delta_pi","phi","gate"]
}

```

1.3 mef_block.json

```

{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"MEFBlock",
  "type":"object",
  "properties":{
    "block_id":{"type":"string"},
    "prev_hash":{"type":"string"},
    "hash":{"type":"string"},
    "timestamp":{"type":"string","format":"date-time"},
    "snapshot_id":{"type":"string"},
    "tic_id":{"type":"string"},
    "route":{"type":"object","properties":{
      "route_id":{"type":"string"},
      "sigma":{"type":"array","items":{"type":"integer"},"minItems":7,"maxItems":7},
      "permutation":{"type":"array","items":{"type":"string","enum":["DK","SW","PI","WT","RES1","ADAPTER","RES2"]},"minItems":7,"maxItems":7},
      "score":{"type":"number"}
    }},
    "proof":{"type":"object","properties":{
      "por":{"type":"string","enum":["valid","invalid"]},
      "phi":{"type":"number"},
      "delta_pi":{"type":"number"},
      "delta_v":{"type":"number"},
      "mci":{"type":["number","null"]}
    }}
  },
  "required":["block_id","prev_hash","hash","timestamp","tic_id"]
}

```

1.4 merkaba_gate.json

```

{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"MerkabaGateEvent",
  "type":"object",
  "properties":{
    "gate_id":{"type":"string"},
    "snapshot_id":{"type":"string"},
    "tic_candidate_id":{"type":"string"},
    "checks":{
      "type":"object",

```

```

    "properties":{"por":{"type":"string","enum":["valid","invalid"]},"
      delta_pi":{"type":"number"},"phi":{"type":"number"},"delta_v":{"
        type":"number"},"mci":{"type":["number","null"]}},
    "required":["por","delta_pi","phi","delta_v"]
  },
  "decision":{"type":"object","properties":{"commit":{"type":"boolean
    "},"reason":{"type":"string"},"required":["commit"]},
  "timestamp":{"type":"string","format":"date-time"}
},
"required":["gate_id","snapshot_id","tic_candidate_id","checks","
  decision","timestamp"]
}

```

1.5 route_spec.json

```

{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"RouteSpec",
  "type":"object",
  "properties":{
    "route_id":{"type":"string"},
    "sigma":{"type":"array","items":{"type":"integer","minimum":1,"maximum
      ":7},"minItems":7,"maxItems":7},
    "permutation":{"type":"array","items":{"type":"string","enum":["DK","
      SW","PI","WT","RES1","ADAPTER","RES2"]},"minItems":7,"maxItems":7},
    "score":{"type":"number"}
  },
  "required":["route_id","sigma","permutation","score"]
}

```

1.6 memory_item.json

```

{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"MemoryItem",
  "type":"object",
  "properties":{
    "id":{"type":"string"},
    "vector8":{"type":"array","items":{"type":"number"},"minItems":8,"
      maxItems":8},
    "sigma":{"type":"object","properties":{"psi":{"type":"number"},"rho
      ":{"type":"number"},"omega":{"type":"number"},"required":["psi","
      rho","omega"]},
    "por":{"type":"string","enum":["valid","invalid"]},
    "tic_id":{"type":"string"},
    "metadata":{"type":"object"}
  },
  "required":["id","vector8","sigma","por","tic_id"]
}

```

2 OpenAPI (Paths+Components, Add-Only)

2.1 Components

```

components:
  schemas:

```

```

SpiralSnapshot: { $ref: "../src/schemas/spiral_snapshot.json" }
TIC:             { $ref: "../src/schemas/tic.json" }
MEFBlock:        { $ref: "../src/schemas/mef_block.json" }
MerkabaGate:     { $ref: "../src/schemas/merkaba_gate.json" }
RouteSpec:       { $ref: "../src/schemas/route_spec.json" }
MemoryItem:      { $ref: "../src/schemas/memory_item.json" }

```

2.2 Paths

`/ping (GET)` returns 200 text.

```

/acquisition:
  post:
    summary: Ingest raw payload
    requestBody: { required: true, content: { application/json: { schema: {
      { type: object } } } } }
    responses:
      "200": { description: ok, content: { application/json: { schema: {
        type: object, properties: { snapshot_id: {type: string} } } } } } }

```

```

/solve:
  post:
    summary: Run S o l v e Coagula to get TIC candidate
    parameters: [{ in: query, name: snapshot_id, required: true, schema: {
      type: string } }]
    responses:
      "200": { description: ok, content: { application/json: { schema: {
        $ref: "#/components/schemas/TIC" } } } } }

```

```

/gate/merkaba:
  post:
    summary: Evaluate Merkaba Gate
    requestBody:
      required: true
      content: { application/json: { schema: { type: object, properties: {
        snapshot_id:{type:string}, tic_candidate_id:{type:string} },
        required: ["snapshot_id","tic_candidate_id"] } } } }
    responses:
      "200": { description: ok, content: { application/json: { schema: {
        $ref: "#/components/schemas/MerkabaGate" } } } } }

```

```

/router/select:
  post:
    summary: Deterministic S7 route selection
    requestBody: { required: true, content: { application/json: { schema: {
      { type: object, properties: { seed:{type:string}, mesh_metrics:{
        type:object} }, required: ["seed","mesh_metrics"] } } } } }
    responses:
      "200": { description: ok, content: { application/json: { schema: {
        $ref: "#/components/schemas/RouteSpec" } } } } }

```

```

/memory/upsert:
  post:
    summary: Insert/update normalized vector into memory
    requestBody: { required: true, content: { application/json: { schema:
      { $ref: "#/components/schemas/MemoryItem" } } } }
    responses: { "200": { description: ok } }

/memory/search:
  post:
    summary: kNN search on normalized vectors
    requestBody: { required: true, content: { application/json: { schema:
      { type: object, properties: { query_vector8: {type: array, items: {
        type: number}, minItems:8, maxItems:8 }, top_k: {type: integer},
        filters: {type: object} }, required: ["query_vector8","top_k"] } }
      } }
    responses: { "200": { description: ok, content: { application/json: {
      schema: { type: object, properties: { hits: { type: array, items: {
        type: object, properties: { id:{type:string}, score:{type:number}
        }, required:["id","score"] } } } } } } } } } } } } } }

/knowledge/derive:
  post:
    summary: Deterministically derive TIC -> Knowledge -> Ledger
    requestBody: { required: true, content: { application/json: { schema:
      { type: object, properties: { inputs:{type: array}, seed_path:{type
        :string}, config:{type:object} }, required:["inputs","seed_path"] }
      } } }
    responses: { "200": { description: ok } }

/knowledge/project:
  post:
    summary: Deterministic projection to a target domain
    requestBody: { required: true, content: { application/json: { schema:
      { type: object, properties: { mef_id:{type:string}, target_domain:{
        type:string}, policy:{type:object} }, required:["mef_id","
        target_domain"] } } } }
    responses: { "200": { description: ok } }

```

3 PlantUML Diagrams

3.1 Component Map

```

@startuml
skinparam packageStyle rectangle
skinparam shadowing false
actor User
package "Domain Layer" { [Adapter] --> (MeshHolo) }
package "MEF-Core" {

```

```

    [Acquisition] -> [Spiral 5D] -> [PoR] -> [ S o l v e Coagula ] -> [Merkaba
        Gate] -> [TIC] -> [HDAG] -> [Ledger]
    [API/CLI]
}
package "Knowledge Layer" {
    [Metric] -> [Memory Index]
    [Knowledge Derive/Project]
}
package "Metatron" { [S7 Adapter] }
(MeshHolo) --> [S7 Adapter]
[S7 Adapter] --> [ S o l v e Coagula ]
[Metric] <- [Spiral 5D]
[TIC] --> [Knowledge Derive/Project] --> [Memory Index]
[API/CLI] --> User
@enduml

```

3.2 Sequence: Derive

```

@startuml
actor Client
Client -> API: POST /knowledge/derive
API -> Acquisition: normalize
Acquisition -> Spiral: S(theta), sigma
Spiral -> PoR: ratios, gap -> valid/invalid
PoR -> Solve: DK/SW/PI/WT (route from Metatron)
Solve -> Gate: delta_pi, phi, delta_v, por
Gate --> Solve: FIRE/HOLD
Gate -> TIC: crystallize (if FIRE)
TIC -> Knowledge: bind seed_path, route, proofs -> mef_id
Knowledge -> Ledger: append block
API --> Client: {knowledge, tic, proof, block}
@enduml

```

4 Agent Execution Checklist (S13–S20)

Preconditions

- Part 1 & 2 all green. All new flags set to `false`. App boots unchanged.

S13 — API Routers (Attach Only)

1. Create flat files: `src__api__memory_endpoints.py`, `src__api__knowledge_endpoints.py`, `src__api__router_metatron.py`.
2. In `src/api/server.py`: `include_router(...)` under `/memory`, `/knowledge`, `/router`.
3. Do not edit existing endpoints.

S14 — Knowledge/Memory Modules

1. Add: `src__knowledge__metric.py` (build $\hat{z}$),
`src__knowledge__index.py` (no-op if `memory.enabled=false`),
`src__knowledge__infer.py` (derive/project/validate pure functions).
2. Keep optional FAISS/hnswlib behind feature flag; no imports if disabled.

S15 — Config Defaults

1. Add keys (if missing):

```
knowledge: { enabled: false }
memory:    { enabled: false }
router:    { mode: inproc }
paths:     { memory: "" }
```

S16 — Tests (Determinism First)

1. Property: same seed/path -> identical $\hat{z}$, route, TIC, ledger hash.
2. Scale invariance: rescale window -> PoR decision unchanged.
3. Route determinism: identical inputs -> identical `route_id`, `sigma`, `permutation`.

S17 — OpenAPI and README

1. Generate OpenAPI from FastAPI; ensure new paths documented; keep legacy text unchanged; add a new section "Knowledge/Memory".

S18 — Docker (Optional)

1. Build container; flags default false; verify app boots identical responses.

S19 — Performance Smoke

1. Measure solve latency and gate pass-rate with small batches; no regression against baseline.

S20 — Release Bundle

1. Pack ZIP with: `README.md`, `requirements.txt`, `pyproject.toml`, `Dockerfile`, `config.yaml`, `docs/`, `src/`.
2. Exclude: `__pycache__`/, `*.pyc`, `.DS_Store`.

5 Security & Non-Interference

- Never log/store BIP-39 mnemonic; only path IDs/hashes.
- New code **must** be add-only; if a core symbol conflicts, create an adapter module instead of editing legacy files.
- New endpoints **must not** alter prior outputs when flags are false.

6 Deterministic Test Matrix (Minimal Commands)

- **Boot**: `uvicorn src.api.server:app -reload`
- **Health**: `GET /ping` → 200
- **E2E x2**: Acquisition → Solve → Gate(FIRE) → Ledger; compare block hashes equal.
- **Router**: same seed+metrics → identical `route_id`, `sigma`, `permutation`.
- **PoR scale**: scale inputs → decision unchanged.

7 Appendix: Materializer Rules (Flat $\rightarrow$ Tree)

- `"__"` maps to folder separators.
- Fix `*.py.py` to `*.py`; replace dashes by underscores.
- Ensure `__init__.py` per `src/*` subdir.

SPEC-006 — MEF Knowledge Engine (Machine-Readable Implementation Blueprint, Part 4)

October 4, 2025

1 Deterministic Primitives (Pseudocode, Normative)

1.1 Canonical JSON (stable serialization)

```
def canonical_json(obj) -> str:
    # RFC8259-compatible, deterministic:
    # - UTF-8, no BOM
    # - sort keys (ASCII order)
    # - floats with 6 fixed decimals
    # - no trailing spaces
    import json, decimal
    class F(json.JSONEncoder):
        def iterencode(self, o, _one_shot=False):
            for s in super().iterencode(o, _one_shot):
                yield s
    def round_floats(o):
        if isinstance(o, float):
            return float(f"{o:.6f}")
        if isinstance(o, dict):
            return {k: round_floats(o[k]) for k in sorted(o.keys())}
        if isinstance(o, list):
            return [round_floats(x) for x in o]
        return o
    o = round_floats(obj)
    return json.dumps(o, separators=(",", ":"), ensure_ascii=False,
        sort_keys=True)
```

1.2 Content Hash (MEF ID) and Block Hash

```
import hashlib

def blake3_hex(data: bytes) -> str:
    # if blake3 not available, use sha256; keep same function name
    try:
        import blake3
        return blake3.blake3(data).hexdigest()
    except Exception:
        return hashlib.sha256(data).hexdigest()

def compute_mef_id(tic: dict, route_id: str, seed_path: str) -> str:
    blob = (canonical_json(tic) + "|" + route_id + "|" + seed_path).encode(
        ("utf-8"))
    return blake3_hex(blob)[:32]

def compute_block_hash(block: dict) -> str:
```

```
body = {k:block[k] for k in block if k != "hash"}
return blake3_hex(canonical_json(body).encode("utf-8"))
```

1.3 HD Seed Derivation (BIP-39 root kept secret)

```
import hmac, hashlib
def derive_seed(root_seed_bytes: bytes, path: str) -> bytes:
    return hmac.new(root_seed_bytes, msg=path.encode("utf-8"),
                    digestmod=hashlib.sha256).digest()
```

1.4 Permutation Space $\mathfrak{S}_7$ and Route Score $J(m)$

```
import itertools, hashlib, json

SLOTS = ["DK","SW","PI","WT","RES1","ADAPTER","RES2"]

def generate_s7():
    return list(itertools.permutations((1,2,3,4,5,6,7), 7))

def mesh_score(m: dict) -> float:
    inv = m.get("invariants", {})
    betti = m.get("betti", inv.get("betti", []))
    lam = m.get("lambda_gap", inv.get("lambda_gap", 0.0))
    pers = m.get("persistence", inv.get("persistence", []))
    b = sum(betti) if isinstance(betti, list) else float(betti or 0.0)
    l = float(lam or 0.0)
    p = sum([(x.get("death",0)-x.get("birth",0)) if isinstance(x,dict)
             else float(x)
             for x in (pers if isinstance(pers,list) else [pers])])
    #  $J(m) = b * b + l + p * p$ 
    return 0.10*b + 0.70*l + 0.20*p

def select_route(seed: str, mesh_metrics: dict) -> dict:
    perms = generate_s7()
    base = seed + ":" + json.dumps(mesh_metrics, sort_keys=True)
    h = int(hashlib.sha256(base.encode("utf-8")).hexdigest(), 16)
    k = int(abs(mesh_score(mesh_metrics))*1000) % len(perms)
    idx = (h + k) % len(perms)
    sigma = perms[idx]
    permutation = [SLOTS[i-1] for i in sigma]
    rid = hashlib.sha256(f"{seed}:{sigma}".encode("utf-8")).hexdigest()
    [:16]
    return {"route_id": rid, "sigma": list(sigma), "permutation":
            permutation, "score": mesh_score(mesh_metrics)}
```

1.5 PI (Path Invariance) — Canonical Representative $\Pi(v)$

```
# Group G of admissible symmetries (permutations and sign-flips) is fixed
# by implementation.
# PI is idempotent and 1-Lipschitz. Deterministic lexicographic
# canonicalization.
from itertools import product, permutations
import numpy as np

def lex_key(vec):
    # tuple with fixed rounding
    return tuple(float(f"{x:.6f}") for x in vec.tolist())
```

```

def generate_G(d: int):
    # Example: only sign flips ( 1 ) and coordinate permutations
    # adjust to your symmetry set.
    for perm in permutations(range(d)):
        for signs in product([-1,1], repeat=d):
            yield (perm, signs)

def apply_g(v: np.ndarray, g) -> np.ndarray:
    perm, signs = g
    w = v[list(perm)].copy()
    return w * np.array(signs, dtype=w.dtype)

def PI(v: np.ndarray) -> np.ndarray:
    best = None
    best_key = None
    for g in generate_G(len(v)):
        w = apply_g(v, g)
        k = lex_key(w)
        if best is None or k < best_key:
            best, best_key = w, k
    return best

```

1.6 Lyapunov ΔV and Alignment Φ

```

import numpy as np

def V(Tv: np.ndarray, v: np.ndarray) -> float:
    return float(np.linalg.norm(Tv - v, ord=2)**2)

def delta_V(Tvt1: np.ndarray, vt1: np.ndarray, Tvt: np.ndarray, vt: np.
ndarray) -> float:
    return V(Tvt1, vt1) - V(Tvt, vt)

def phi(v_next: np.ndarray, Tv: np.ndarray) -> float:
    a = float(np.linalg.norm(v_next, ord=2))
    b = float(np.linalg.norm(Tv, ord=2))
    if a == 0.0 or b == 0.0: return 0.0
    return float(np.dot(v_next, Tv) / (a*b))

```

2 Example Config and Feature Flags (Add-Only)

2.1 config.yaml (fragment)

```

seed: "MEF_SEED_42"

spiral: { a: 1.0, b: 0.2, c: 0.2, k: 2, step: 0.01 }
por:    { psi_min: 0.10, psi_max: 0.85, lambda_gap_star: 0.15, snr_star:
6.0 }

operators:
  dk: { alpha1: 0.05, alpha2: 0.03, clip: 1.5 }
  sw: { tau: 0.10 }
  pi: { canon: "lex", tol: 1.0e-6 }
  wt: { gamma: 0.10 }

solve: { lambda: 0.8, eps_fix: 1.0e-6, max_iter: 1000 }

```

```

gate:  { eps: 1.0e-6, phi_star: 0.60, mci_min: 0.85 }

knowledge: { enabled: false }
memory:    { enabled: false }
router:    { mode: inproc }
paths:     { memory: "" }

```

3 Example Payloads (Copy-Paste Ready)

3.1 /acquisition (POST)

```

{
  "payload": {"text": "example"},
  "seed_path": "MEF/text/spiral/0001"
}

```

3.2 /solve (POST ?snapshot_id=...)

```

{
  "tic_id": "TIC-9f2a",
  "snapshot_id": "S-1a2b",
  "delta_pi": 0.000004,
  "phi": 0.72,
  "gate": {"por": "valid", "fire": false},
  "invariants": {"symmetry": "C2", "entropy": 0.431},
  "timestamp": "2025-10-05T12:00:00Z"
}

```

3.3 /gate/merkaba (POST)

```

{
  "gate_id": "G-0a1b",
  "snapshot_id": "S-1a2b",
  "tic_candidate_id": "TIC-9f2a",
  "checks": {"por": "valid", "delta_pi": 0.000004, "phi": 0.72, "delta_v": -0.11, "mci": null},
  "decision": {"commit": true, "reason": "all_thresholds_passed"},
  "timestamp": "2025-10-05T12:00:02Z"
}

```

3.4 RouteSpec

```

{
  "route_id": "7f1e0c9b4f2a1d3c",
  "sigma": [3,1,4,2,7,5,6],
  "permutation": ["DK", "SW", "PI", "WT", "RES1", "ADAPTER", "RES2"],
  "score": 0.842
}

```

3.5 Knowledge Object (derive result)

```

{
  "knowledge": {
    "mef_id": "a8b3c7d2e601ab90c1f02d43cdab7712",

```

```

"tic": {"tic_id":"TIC-9f2a","invariants":{"symmetry":"C2","entropy":0.431}},
"proofs":{
  "por":"valid","phi":0.72,"delta_pi":0.000004,"delta_v":-0.11,
  "route":{"route_id":"7f1e0c9b4f2a1d3c","sigma":[3,1,4,2,7,5,6],
    "permutation":["DK","SW","PI","WT","RES1","ADAPTER","RES2"],
    "score":0.842}
},
"seed_path":"MEF/text/spiral/0001",
"context":{"hdag_refs":[],"parents":[],"children":[]}
},
"ledger_block": 42
}

```

3.6 /memory/upsert and /memory/search

```

{
  "id":"MEF-a8b3",
  "vector8":[0.2231,0.0000,0.1076,0.0981,0.5348,0.1180, -0.2144,0.7382],
  "sigma":{"psi":0.31,"rho":0.28,"omega":0.41},
  "por":"valid",
  "tic_id":"TIC-9f2a",
  "metadata":{"domain":"text"}
}

{
  "query_vector8":
    "[0.2200,0.0001,0.1100,0.1000,0.5300,0.1200,-0.2100,0.7400]",
  "top_k": 5,
  "filters": {"domain":"text"}
}

```

4 Deterministic Vector Build (Reference)

```

import numpy as np

def build_vector8(x5, sigma, w):
    # x5 = [x1..x5], sigma = {psi,rho,omega}, w = {w1..w5,wpsi,wrho,womega}
    }
    z = np.array([
        w["w1"]*x5[0], w["w2"]*x5[1], w["w3"]*x5[2], w["w4"]*x5[3], w["w5"]
        ]*x5[4],
        w["wpsi"]*sigma["psi"], w["wrho"]*sigma["rho"], w["womega"]*sigma[
        "omega"]
    ], dtype=float)
    n = float(np.linalg.norm(z, ord=2))
    return (z / n).tolist() if n > 0 else z.tolist()

```

5 Red List (MUST NOT change under any circumstance)

1. **Do not** change DK, SW, PI, WT semantics or parameter meanings.
2. **Do not** alter PoR decision thresholds in existing code; only read from config. If absent, add defaults without overwriting.
3. **Do not** change Solve-Coagula iteration scheme, stop rule keys, or their names.

4. **Do not** rename or remove fields in existing JSON schemas (`spiral_snapshot.json`, `tic.json`, `mef_block.json`).
5. **Do not** modify existing endpoints; new endpoints must live under `/router`, `/memory`, `/knowledge`.
6. **Do not** persist the BIP-39 mnemonic or raw seeds; only path IDs/hashes.
7. **Do not** change ledger hash computation or float precision policy once set.
8. **Do not** introduce nondeterminism (no `random()`, no time-based seeds) in core paths.
9. **Do not** change timestamp format; keep ISO-8601 UTC with Z.
10. **Do not** mutate old blocks; ledger is append-only.

6 Yellow List (Allowed via Adapters/Shims only)

- Missing imports like `ingestion.triton`: add shim file `triton.py` that re-exports from `triton_core`.
- Metatron access: add adapter under `topology/` or `metatron_vendor/`, never edit core.
- Memory index: implement behind `memory.enabled` flag; no-op if disabled.

7 Error Handling (Deterministic)

- If PoR invalid: return HOLD event, no ledger append.
- If Gate fails: return HOLD with `phi`, `delta_pi`, `delta_v`, route proof attached.
- If router service unreachable and `router.mode=service`: fallback to inproc adapter; log event.
- If index path missing and `memory.enabled=true`: fail the request with 400; do not alter core state.

8 Test Vectors (Fixed)

8.1 Vector Equivalence (Cosine = L2 on normalized)

```
import numpy as np
z = np.array([0.2,0.0,0.1,0.1,0.53,0.12,-0.21,0.74])
z = z/np.linalg.norm(z)
y = np.array([0.22,0.0,0.11,0.1,0.53,0.12,-0.21,0.74]); y = y/np.linalg.
    norm(y)
cos = float(np.dot(z,y))
l2 = float(np.linalg.norm(z-y)**2 / 2.0)
assert abs(cos - (1 - l2)) < 1e-9
```

8.2 PoR Scale Invariance

```
def por_decision(window, sigma):
    # agent shall plug in the actual PoR ratio/gap computation
    return "valid"
w1 = [0.5*x for x in range(10)]
w2 = [2.0*x for x in range(10)]
assert por_decision(w1, {"psi":0.3,"rho":0.2,"omega":0.5}) == \
    por_decision(w2, {"psi":0.3,"rho":0.2,"omega":0.5})
```

8.3 PI Idempotence

```
import numpy as np
v = np.array([0.3,-0.1,0.2])
w = PI(v)
u = PI(w)
assert np.allclose(w,u) # idempotent
```

9 Operational Notes

9.1 OpenAPI Exposure

Agent SHALL expose schemas via FastAPI `app.openapi()`, ensuring new components/paths appear without touching legacy ones.

9.2 Packaging

- Include: `README.md`, `requirements.txt`, `pyproject.toml`, `Dockerfile`, `config.yaml`, `docs/`, `src/`
- Exclude: `__pycache__`/, `*.pyc`, `.DS_Store`

9.3 Acceptance Gates

- With all new flags OFF: responses byte-for-byte identical to pre-integration on the legacy endpoints.
- With flags ON: determinism tests pass; two identical runs yield identical ledger hash for the final block.

End of SPEC-006 (Parts 1–4)

This blueprint is sufficient for an automated agent to implement the add-only Knowledge/Memory extension deterministically without altering the MEF-Core.